DBMS PROJECT-BASED LEARNING

Bank Account Management System

MySQL | Python | MySQL Connector | VS Code

Presented by: Souri Keerthana(25WU0104026)

B.tech(AI and ML) - Leopards

Faculty:

Problem Statement & Objectives

The Problem

Manual banking data handling leads to errors, duplication and difficulty retrieving information.

Solution: a centralized MySQL database with a Python application

Objectives

1
Manage customer & account information

2
Record deposits, withdrawals & transfers

3
Manage loans & repayments

4
Maintain integrity using constraints

5
Generate reports using SQL

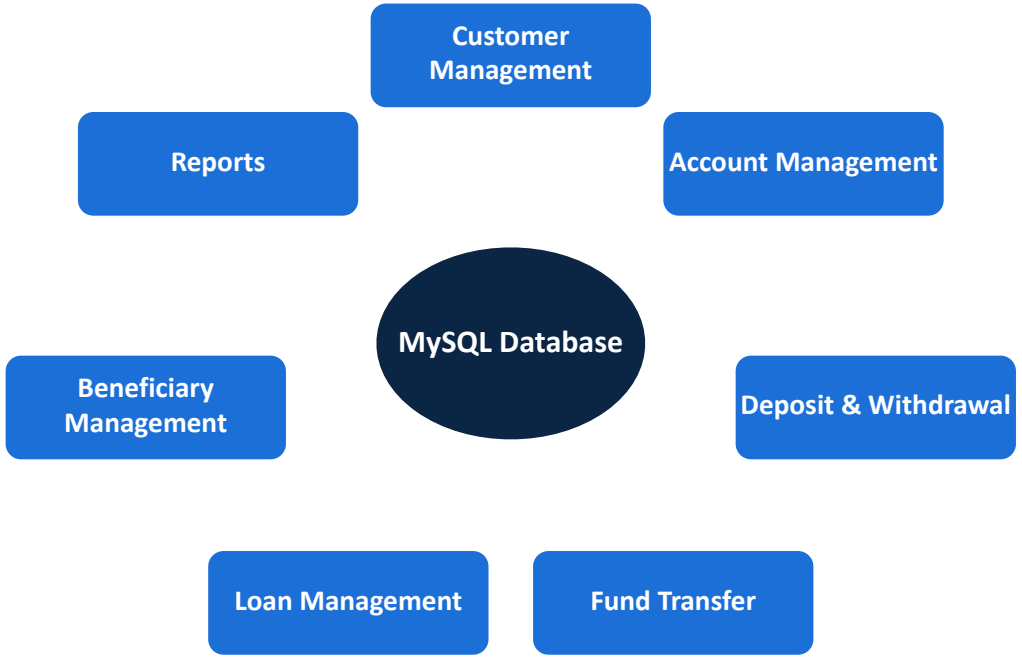
6
Python-based database application

Proposed System & Scope

A centralized database-driven banking system where a Python application manages banking data in MySQL.

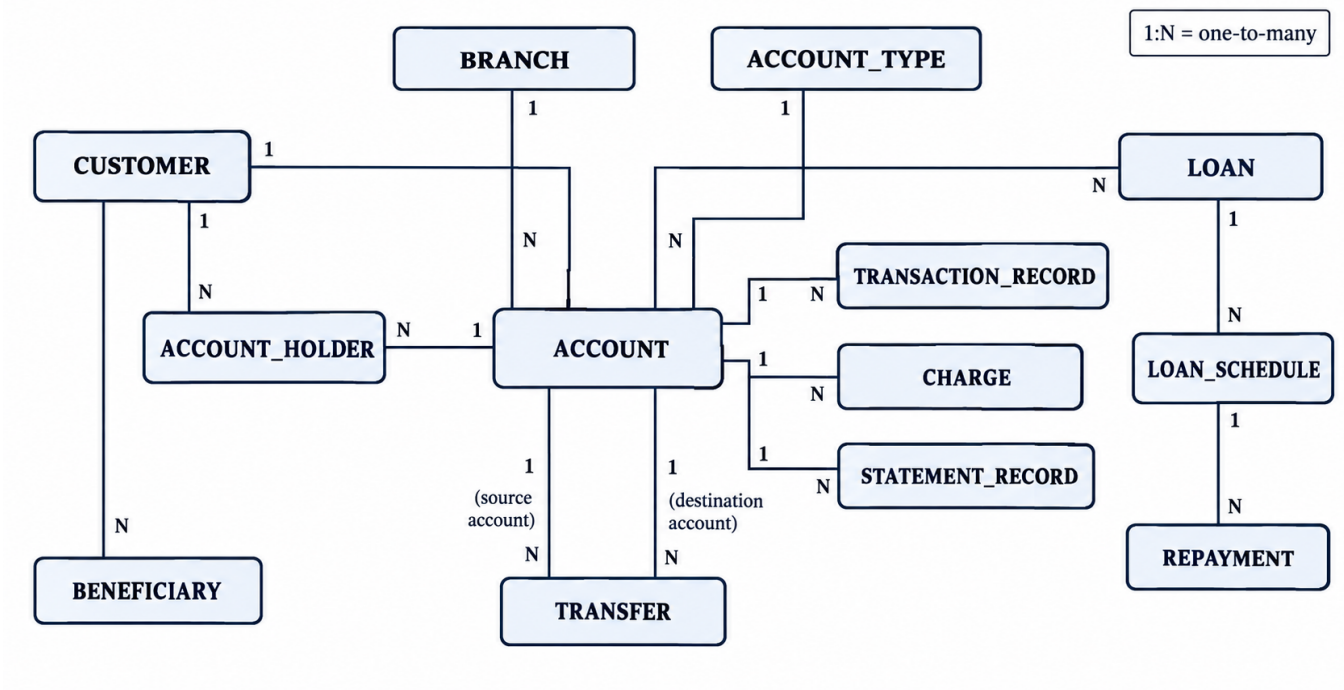
Scope

Account and transaction management, with basic loan and reporting functionality.



ER Diagram & Database Design

13 entities: Customer · Branch · Account Type · Account · Account Holder · Transaction Record · Transfer · Beneficiary · Charge · Loan · Loan Schedule · Repayment · Statement Record



Relational Schema, Keys & Constraints

CUSTOMER	ACCOUNT	ACCOUNT_HOLDER
• customer_id — PK • phone — UNIQUE • email — UNIQUE	• account_id — PK • account_number — UNIQUE • account_type_id — FK • branch_id — FK	• account_id + customer_id • Composite PK

Constraints Used

Primary Key | **Foreign Key** | **UNIQUE** | **NOT NULL** | **CHECK**

Constraints maintain data integrity and consistency.

Functional Dependencies & Normalization

Functional Dependencies (examples)

- customer_id → first_name, last_name, phone, email, address, date_of_birth
- branch_id → branch_name, branch_code, address, phone
- account_id → account_number, account_type_id, branch_id, balance, status
- loan_id → customer_id, loan_type, principal_amount, interest_rate, start_date, end_date, status

Normalized up to 3NF



Atomic values, no repeating groups

1NF + no partial dependency

2NF + no transitive dependency

Reduces redundancy and update, insertion & deletion anomalies

Database Implementation

MySQL · bank_management · 13 tables

8	Customers	8	Accounts
3	Branches	10	Transactions
3	Transfers	3	Loans
5	Loan Schedules	2	Repayments

```
mysql> USE bank_management;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+----------------------------+
| Tables_in_bank_management |
+----------------------------+
| account                    |
| account_holder             |
| account_type               |
| beneficiary                |
| branch                     |
| charge                     |
| customer                   |
| loan                       |
| loan_schedule              |
| repayment                  |
| statement_record           |
| transaction_record         |
| transfer                   |
+----------------------------+
13 rows in set (0.005 sec)
```

SQL Queries & Reports

- Customer details
- Account details with customer info
- Account transaction history
- Transfer report
- Customer loan report
- Pending loan installments
- Branch-wise account summary
- Deposit & withdrawal summary

SQL Concepts

SELECT | JOIN | WHERE | GROUP BY

ORDER BY | COUNT() | SUM()

Example: Branch-wise Account Summary — Branch Name | Total Accounts | Total Balance

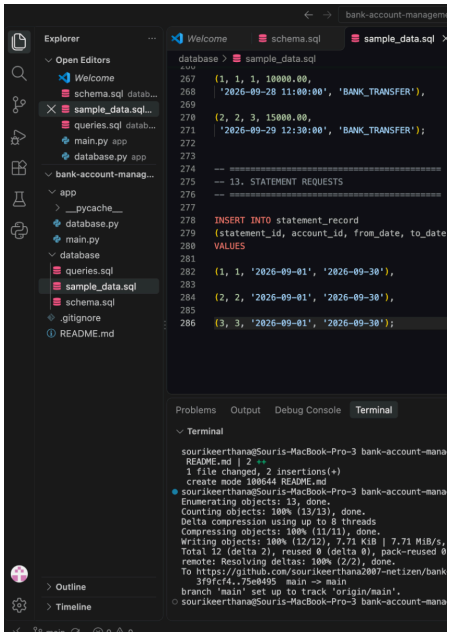
```
mysql> SELECT
    ->     b.branch_name,
    ->     COUNT(a.account_id) AS total_accounts,
    ->     SUM(a.balance) AS total_balance
    -> FROM branch b
    -> LEFT JOIN account a
    ->     ON b.branch_id = a.branch_id
    -> GROUP BY b.branch_id, b.branch_name;
+----------------------+----------------+---------------+
| branch_name          | total_accounts | total_balance |
+----------------------+----------------+---------------+
| Hyderabad Main Branch |              3 |     133700.00 |
| Secunderabad Branch   |              2 |      87500.00 |
| Gachibowli Branch     |              3 |     104500.00 |
+----------------------+----------------+---------------+
3 rows in set (0.005 sec)
```

Application Architecture & Connectivity



Technologies

- Interface: Python console application
- Backend: MySQL database
- Connectivity: MySQL Connector for Python
- Connects to bank_management and runs SQL directly



Application Modules & CRUD

Main Menu

Customer Management

View · Add · Update

Account Management

View account details

Transactions

Deposit · Withdrawal

Transfers

Account-to-account

Loan Management

View loan information

Reports

Customer, transaction, transfer, loan, branch

Validation

Required customer info · 10-digit phone · Positive amount · Account exists · Sufficient balance · Different source & destination accounts

Testing & Application Screens

Test Case	Expected	Result
View customers	Records displayed	Pass
Add valid customer	Customer added	Pass
Invalid phone number	Error displayed	Pass
Deposit	Balance increases	Pass
Withdrawal	Balance decreases	Pass
Insufficient balance	Transaction rejected	Pass
Transfer	Both balances updated	Pass
View reports	Report generated	Pass

Conclusion & Future Scope

Conclusion

- Structured data storage
- Integrity through constraints
- Transaction & loan management
- SQL-based reports
- Python–MySQL connectivity
- Input validation

Future Scope

- Graphical User Interface
- Authentication & role-based access
- Online banking integration
- Mobile application
- Analytics dashboards
- Enhanced security & encryption

Thank You

REPORT LINK

https://drive.google.com/file/d/1p4D5mrtNTjv07D3LEeXIdt5Hz7teOAKu/view?usp=drive_link

Git Hub link

https://github.com/sourikeerthana2007-netizen/bank-account-management.git